Building a Unix Shell: Freshman to Senior Guide

Oct 10, 2026 · @Nathan

Your HW3 shell is a read–parse–run loop built on three system calls (fork, exec, wait) and one trick (rewiring file descriptors); this guide explains it at four depths, first-year to senior, using the corrected code in assignment3_fixed.zip.

The assignment in plain English

You are writing a *shell*: the program that prints a prompt, reads a line you type, and starts other programs for you. Bash is a shell, and yours is a miniature one. The handout (Homework 3, "Execution Integrity and Do Process") asks for these features:

- **Run programs.** `ls -l` starts the program `ls` as a separate process, and the shell waits until it finishes before showing the next prompt.
- **Sequences.** `a ; b` runs `a`, then `b`. `a & b` starts `a` in the *background*, so the shell moves on to `b` immediately.
- **Pipelines.** `a | b` connects the output of `a` to the input of `b`, so both run at the same time.
- **Redirection.** `cmd < in > out` reads input from a file and writes output to a file.
- **Builtins.** `exit`, `pwd`, `cd` (including `cd -`), and `history` are handled by the shell itself, not by an outside program.

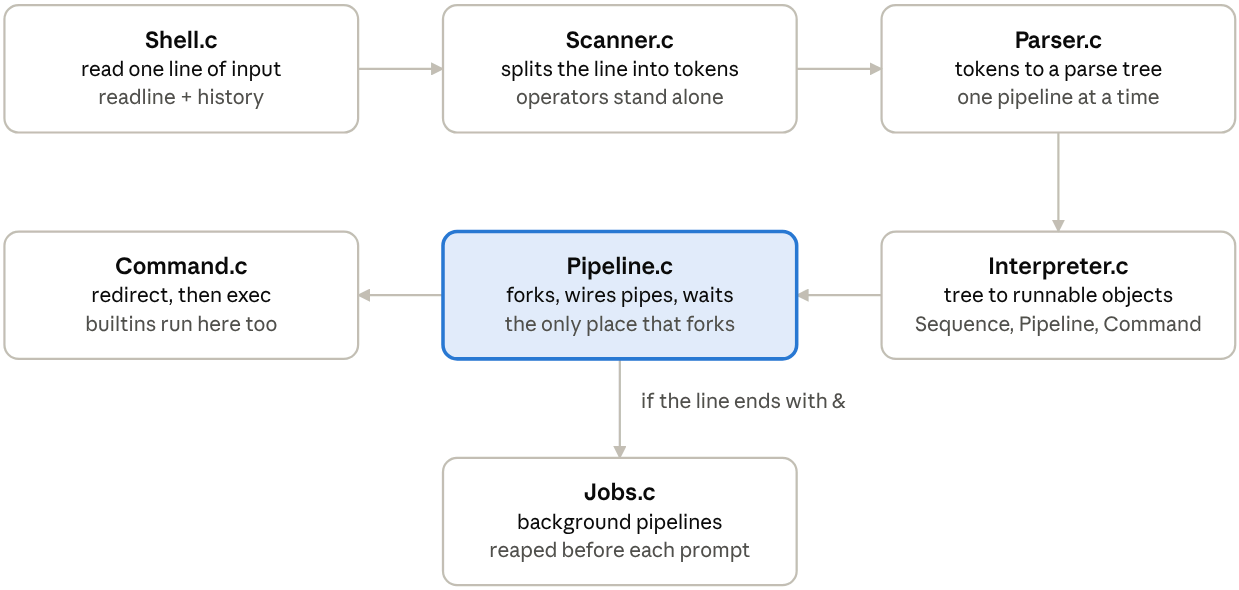
Two rules about *how* it works matter more than the features. Builtins run inside the shell's own process unless they are backgrounded or in a pipeline. Background children must eventually be collected so they do not become "zombies" (dead processes nobody has acknowledged).

The handout also grades software engineering: modularity ("your grade will depend on good modularity"), a regression test suite run with `Test/run`, a valgrind-clean program, and your own deque from HW1. The CS 552 extras (subshells, `{ }` groups, `jobs`, `fg`, `bg`) are not part of the 452 version, and this shell does not implement them.

You were given a skeleton: a scanner, a mostly complete parser, and a tree-walking interpreter whose execution functions were stubs. Your work is the part that actually forks, wires up pipes, and waits.

The big picture

One typed line passes through six modules, and only Pipeline.c forks



Highlighted box: the only module that calls fork().
A lone foreground builtin (cd, pwd, ...) skips the fork: Pipeline asks Command to run it inside the shell.
deq.c supplies the lists inside Sequence, Pipeline and Jobs.

module flow · 7 modules, 1 fork point

Read it as a snake: the top row runs left to right, then it drops down and runs right to left. Scanner, Parser and Interpreter only *describe* the work. Pipeline.c does the forking, and Command.c then does its part inside each child (or inside the shell itself, for a lone builtin).

Freshman level: processes, fork, exec, wait

Program vs. process. A *program* is a file on disk, such as `/bin/ls`. A *process* is a running copy of a program, with its own memory, its own number (the PID), and its own set of open files. You can run the same program in ten processes at once.

The shell is a loop. Everything in `Shell.c` is this idea:

```
while (not told to exit):
    line = read a line from the user
    parse the line into commands
    run the commands
```

Running a program takes three system calls. A system call is a request to the operating system that your program cannot do by itself.

1. `fork()` makes a near-identical copy of the calling process. It is called once but returns twice: in the original (the *parent*) it returns the copy's PID, and in the copy (the *child*) it returns 0.
2. `exec()` replaces the calling process's program with a different one. If it works, it never returns, because the old program no longer exists. (We use `execvp`, which also searches your `PATH` for the program.)
3. `waitpid()` makes the parent sleep until a child exits.

A way to picture it: `fork` is a photocopier that copies you, `exec` is the copy being retrained to do a different job, and `wait` is the original standing at the door until the copy comes back. This is the whole pattern in miniature:

```
int pid = fork();
if (pid == 0) {                  // child
    execvp(argv[0], argv);       // become the program, e.g. ls
    _exit(127);                  // only reached if exec failed
} else {                         // parent (the shell)
    waitpid(pid, NULL, 0);       // wait for the child to finish
}
```

Why not just run `ls` directly? If the shell called `exec` itself, the shell would be *replaced* by `ls` and would vanish when `ls` finished. Forking first sacrifices a copy so the shell survives.

Input and output. Every process starts with three open "files": number 0 is input (`stdin`, the keyboard), number 1 is output (`stdout`, the screen), and number 2 is error messages (`stderr`). Redirection (`<`, `>`) and pipes (`|`) work by changing what those numbers point to. The program itself does not know or care.

Why `cd` must be built in. The current directory belongs to a *process*. If the shell ran `cd` in a child, the child would change directory and then exit, and the shell would not move. So `cd` (and `exit`) must run inside the shell. The same reasoning explains `cd /etc | pwd`: a command in a pipeline runs in a child, so its `cd` is lost and `pwd` still prints the old directory.

Background jobs. `cmd &` means "fork the same way, but skip the wait." The child is still your responsibility. When it finishes it stays in the process table as a *zombie* until the parent collects its exit status with `wait`. A pile of zombies is a resource leak, which is why

the handout insists that background children eventually be waited for.

Sophomore level: the code, data structures, and memory

The modules. Each `.c` file does one job and starts with a comment saying what it is. Reading them in this order works well:

File	Job	Main data structure
`Shell.c`	`main()`: read a line, run it, repeat	the `eof` flag and a `Jobs` set
`Scanner.c`	Split a line into tokens (`echo`, `a`, `;`, …)	a string and a cursor
`Parser.c`	Tokens to a parse tree, one function per grammar rule	the tree in `Tree.h`
`Interpreter.c`	Parse tree to runnable objects	`Sequence` of `Pipeline` of `Command`
`Sequence.c`	Run pipelines one after another	a deque of pipelines
`Pipeline.c`	Fork one child per command, connect them with pipes, wait	a deque of commands plus child PIDs
`Command.c`	One command: builtins, redirection, `exec`	an `argv` array
`Jobs.c`	Track background pipelines, reap finished children	a deque of pipelines
`deq.c`	Double-ended queue, used by all of the above	a doubly linked list

Headers and information hiding. A `.h` file is the public face of a module and the `.c` file is the private inside. Here `typedef void *Pipeline;` in `Pipeline.h` hides the layout from everyone else. Inside `Pipeline.c`, each function casts the handle back to the real struct (`PipelineRep`). Other modules can only call the functions, so you can change the struct without breaking them. This is how C imitates a Java `private` field.

The deque. A deque is a list you can add to or remove from at both ends. Ours is a doubly linked list: each node holds its data plus two pointers, `link[Head]` (previous) and `link[Tail]` (next). Because the code is written for "an end `e`" rather than "the head" or

"the tail", one `put` function serves both `deq_head_put` and `deq_tail_put`. Do you see why `OTHER(e)` is `1 - e`? It flips 0 and 1.

Two trees of objects. The *parse tree* (`T_sequence`, `T_pipeline`, `T_command`, ...) only describes what the user typed. The *runnable objects* (`Sequence`, `Pipeline`, `Command`) know how to run it and carry extra state such as child PIDs. Keeping them separate lets the parser be thrown away after every pipeline.

Who frees what. C has no garbage collector, so every `malloc` needs exactly one `free`, and the cleanest way to keep that straight is a rule about ownership:

- The parse tree owns its strings, and `Parser.c` frees the tree after each pipeline runs.
- `newCommand` makes its *own copies* of the words and file names, so it never depends on the tree staying alive.
- A `Pipeline` owns its `Command`s, and a `Sequence` owns its `Pipeline`s.
- `Jobs` owns background `Pipeline`s until their children are reaped.

In the first version, the tree and the `Command` shared the redirection strings and had to agree about who freed them. Copying removes that coupling. `valgrind` confirms there are no leaks, apart from a few terminal-library blocks it reports as still reachable (3.5 KB in my run).

Why `argv` ends in NULL. `execvp(file, argv)` takes an array of C strings and finds its end by looking for a `NULL` pointer, since it has no length parameter. `newCommand` counts the words, allocates `n + 1` slots, and sets the last one to `NULL`.

The Makefile. `make` rebuilds a file only when something it depends on is newer. The `-MMD` flag makes gcc write `.d` files listing each object's header dependencies, and `-include` reads them. Without that, editing `Command.h` would not rebuild `Pipeline.o` and you would chase ghosts. `-lreadline -lncurses` links the libraries that provide line editing and history.

Readline and history. `readline(prompt)` returns a malloc'd line with arrow-key editing, and returns `NULL` on Ctrl-D. The GNU History library remembers lines (`add_history`) and exposes them through `history_list()`, which the `history` builtin prints.

Junior level: file descriptors, pipes, and parsing

File descriptors and `dup2`

A *file descriptor* is a small integer that indexes a per-process table of open files. `open()` returns the lowest unused number, and numbers 0, 1, 2 are `stdin`, `stdout`, `stderr`. They survive `fork` (the child gets copies of the table) and survive `exec` (the new program inherits them). That is the whole trick behind redirection: the shell arranges the table *after* `fork` and *before* `exec`, and the new program just uses 0 and 1 without knowing where they lead.

`dup2(old, new)` makes descriptor `new` refer to the same open file as `old`, closing whatever `new` referred to. Redirecting output is therefore three steps, shown here from `redirect()` in `Command.c`:

```
int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, 1);     // descriptor 1 (stdout) now leads to out.txt
close(fd);       // the extra name is no longer needed
```

A builtin that runs *inside the shell* must not leave stdout redirected afterwards. `runBuiltin` therefore saves copies with `dup(0)` and `dup(1)` first, runs the builtin, flushes, and restores them with `dup2`.

Pipes

`pipe(p)` creates a kernel buffer with two descriptors: `p[1]` to write into and `p[0]` to read from. For `a | b | c` the shell creates two pipes and three children:

```
  a  --stdout--> [pipe 0] --stdin-->  b  --stdout--> [pipe 1] --stdin-->  c
       fd 1 = p0[1]          fd 0 = p0[0]   fd 1 = p1[1]          fd 0 =
p1[0]
```

Three behaviors of pipes decide whether your code works:

- Reading from an empty pipe *blocks* (waits), and writing to a full one blocks. Linux buffers about 64 KB, which is why `Test_pipe_big` pushes 1 MB through three stages.
- A reader sees end-of-file only when *every* copy of the write end is closed, in every process. That includes the shell's own copies and the sibling children's copies.
- Writing to a pipe with no readers raises `SIGPIPE`, which kills the writer. That is how `yes | head -n 2` stops: `head` exits, then `yes` is killed on its next write.

The rule that follows is the one students most often get wrong: **after `dup2`, every process closes every pipe descriptor it does not need.** In `Pipeline.c`, each child does its two `dup2` calls and then `closeAll`; the parent calls `closeAll` right after forking. If any process keeps a stray write end, `cat` in `a | cat` waits forever for an EOF that never comes.

Order matters at one more spot. A pipe is connected first and a `<` or `>` is applied after it, so a redirection overrides the pipe. In `a > f | b`, the output goes to `f` and `b` immediately sees an empty input.

Waiting, background jobs, and zombies

`waitpid(-1, ...)` means "any child". A foreground pipeline loops on it until all of its own children have been collected. If it happens to collect a *background* job's child instead, `creditJobs` records that against the right job rather than losing it. Before each prompt, `reapJobs` calls `waitpid(-1, NULL, WNOHANG)` in a loop; `WNOHANG` means "do not sleep if nothing has finished." `exit` finishes with `drainJobs`, which sleeps until every background child is done, as the handout requires.

From text to tree: scanner, grammar, parser

The handout gives the language as a *grammar*, a set of rewrite rules:

```
sequence ::= pipeline | pipeline & | pipeline ; | pipeline & sequence |
pipeline ; sequence
pipeline ::= command  |  command "|" pipeline        (the quoted | is the
pipe character)
command  ::= words redir
redir    ::= (empty) | < word | > word | < word > word
```

The **scanner** cuts the line into tokens. An operator (`| & ; < >`) is always its own token, so `echo a;ls>out` becomes `echo a ; ls > out`. The **parser** is *recursive descent*: one function per rule, each deciding what to do by looking at only the current token. `eat("|")` means "if the current token is `|`, consume it." For `ls | wc > out &`:

```
sequence (op "&")
  pipeline
    command: words [ls]
    pipeline
      command: words [wc], redir out = "out"
```

The grammar also encodes precedence without anyone writing it down. `|` binds tighter than `;` and `&`, and a redirection belongs to the single command next to it.

One design choice deserves attention. `interpret()` parses *one pipeline, runs it, then parses the next*, rather than parsing the whole line first. That is required for correctness: in `cd /etc ; pwd`, the `cd` must have happened before `pwd` runs, and in `exit ; echo hi` the `echo` must not run. It also means that in `echo a ; echo b >`, the first command has already run when the syntax error in the second is found. Bash would reject the whole line first, and this shell does not.

Senior level: design, correctness, and limits

Interface design: who knows what

In the first version, `Command.c` received the whole array of pipes and decided which ones to keep, so a module about *one command* knew the *topology of the pipeline*. In this version `Pipeline.c` owns the topology (it creates pipes, forks, and does the `dup2` calls) and `Command.c` exposes only `execChild(cmd)`, which applies redirections and runs. Dependencies now point one way: `Pipeline` uses `Command`, never the reverse. That is the kind of change a handout calling for "good modularity" is looking for, and it removed about 40 lines of fd bookkeeping.

Three invariants that make it correct

1. **Every pipe end is closed in every process that does not use it.** Violations cause hangs, not crashes, so they are hard to find by testing alone.
2. **Every child is reaped exactly once.** Too few waits leave zombies. Crediting a PID to the wrong job, or twice, corrupts the pending count and can free a job early.
3. **The shell's own `stdin` and `stdout` are never left redirected.** `runBuiltin` restores them on every path, including when the redirection itself fails.

Subtle details worth knowing

- **`fflush(NULL)` before `fork`, and `_exit` in children.** `fork` duplicates the stdio buffers. If the parent had unflushed output, both processes would eventually flush a copy of it and the text would appear twice. Children also leave with `_exit`, which

skips the `atexit` handlers and stdio flushing that `exit` runs; those belong to the parent's state. (The one exception is a builtin run in a child, which flushes its *own* output explicitly before `_exit(0)`.)

- **`waitpid(-1)` is a single shared reaper.** Waiting for specific PIDs would also work, but any background child that finished while you waited would then linger as a zombie. With one reaper plus `creditJobs`, nothing is dropped. The cost is that exit statuses are discarded: there is no `$?`, and a pipeline's status would conventionally be its last stage's.
- **Redirect failure in a pipeline stage.** If `cat < missing | wc` cannot open `missing`, that child exits with status 1 and `wc` sees an empty input. That matches bash's behavior.
- **Complexity.** `deq_head_ith` is O(n), so loops over a deque are O(n^2). With a handful of commands or jobs this is irrelevant, and a clearer loop is worth more than a faster one here.
- **Why `execvp` and not `system()`.** `execvp` receives the arguments as an array and the kernel starts the program with exactly those strings, so nothing is parsed a second time. `system()` would hand one string to `/bin/sh` to parse again, which would make your shell a thin wrapper around another shell and open the door to command injection.

Known limitations (and what a fix involves)

- **Zombies linger until the next Enter.** `reapJobs` runs before each prompt. A background child that finishes while the shell is blocked in `readline` stays a zombie until you press a key. Real shells install a `SIGCHLD` handler. It must be async-signal-safe (set a flag, or write to a self-pipe), and the main loop needs `sigprocmask` or `pselect` to avoid a lost-wakeup race.
- **Ctrl-C kills the shell during a foreground command.** I verified this on a pseudo-terminal, in both your original and the corrected shell. The terminal sends `SIGINT` to the whole *foreground process group*, which includes the shell, and the shell has the default (fatal) handler outside `readline`. Fixing it properly is the CS 552 job-control topic: ignore `SIGINT`, `SIGTSTP` and `SIGTTOU` in the shell, put each job in its own group with `setpgid`, hand it the terminal with `tcsetpgrp`, reset signals to default in the child before `exec`, and use `WUNTRACED` and `SIGCONT` for `fg` and `bg`.
- **No quoting, variables, or globbing.** The scanner splits on whitespace, so a filename with a space cannot be written. A real shell has an expansion phase (variables, globs, quotes) between scanning and execution.

- **Fatal errors are literal.** `fork`, `pipe`, and `malloc` failures call `ERROR`, which exits the shell. That is defensible here. A production shell would clean up the half-built pipeline and report the failure.

How the tests were checked

A passing test only proves something if it can fail. I ran the new tests against *your original* binary: the bare-`cd` test segfaulted it, the syntax-error test killed it, and the `cd -` test printed 2 instead of 1. That is how I discovered that my first draft of the `cd -` test passed on the buggy shell, because calling `pwd` first quietly initialised the state the bug depended on. I changed the test to start with the external `/bin/pwd`. I also ran the suite 30 times to check for flakiness (0 failures), and ran valgrind on every test (0 errors, 0 definite leaks).

The suite deliberately omits `pwd & cd /etc ; pwd`. One of its two lines is `/tmp` and the other is `/etc`, and their order depends on process scheduling, so any fixed expected-output file would sometimes fail.

How real shells differ

Bash forks per pipeline stage much as this shell does, but it also tracks process groups and job tables, expands words, and handles signals. Some shells (zsh, ksh) run the *last* stage of a pipeline in the current shell, so `echo x | read v` can set a variable there; this shell, like bash by default, does not.

The handout's examples, traced

The handout lists six inputs and says to "test your pipelines carefully". Each row below starts with the shell in `/tmp`. I ran every one on the corrected shell, and the output column is what it printed.

Input	Output	Why
`cd /etc ; pwd`	`/etc`	A lone foreground `cd` is a builtin, so it runs in the shell and the shell's directory really changes.

Input	Output	Why
`cd /etc | pwd`	`/tmp`	Two commands make a pipeline, so both run in children. The child's `chdir` dies with the child, and the shell never moves.
`pwd | cat ; cd /etc | pwd`	`/tmp`, `/tmp`	Both pipelines run entirely in children, so the shell stays in `/tmp` throughout.
`pwd & cd /etc & pwd`	`/tmp`, `/tmp`	All three are backgrounded, so each runs in a child; the shell itself never changes directory. The two `pwd` lines may print in either order, but both read `/tmp`.
`pwd ; cd /etc & pwd`	`/tmp`, `/tmp`	The `cd` is backgrounded, so it runs in a child and has no effect on the shell.
`pwd & cd /etc ; pwd`	`/tmp` and `/etc`, in either order	The first `pwd` is a background child forked while the shell was still in `/tmp`. The `cd` is a foreground builtin, so the shell moves, and the second `pwd` prints `/etc`. The two outputs race.

The lesson is that *where a command runs* (shell or child) decides whether it can change the shell's state, and that is determined by only two facts: is it a lone builtin, and is it in the foreground? The last row also shows that output from background jobs interleaves unpredictably, which is why an automated test cannot pin down its order.

What changed from your submission, and why

The first evaluation found two real bugs, one misleading test, and one test that could only pass on your own machine. The rest of the changes are simplifications.

#	Symptom	Root cause	Fix
1	`cd` with no argument segfaults	`builtin_args` only printed a warning and returned, so `bi_cd` went on to call `strcmp` on a NULL `argv[1]`	`checkArgs` returns success or failure and every builtin stops on failure
2	`cd -` fails the first time it is used	The shell cached its directory in `cwd`, but only `pwd` filled it in. The first `cd` stored NULL as the "previous" directory	No cache: `cd` asks `getcwd()` each time, before changing directory
3	`Test_cd_dash` passed despite bug 2	It ended with `cd /tmp ; pwd`, which hid the failure, and `Test/run` does not compare stderr, where the warning went	Test returns to the previous directory and prints it; a second test checks the first-`cd` case
4	`Test_pwd` fails on any other machine	Its expected output contained a path from your computer	The test runs `cd /tmp` first
5	A typo like `echo a >` exits the shell	The parser's error macro called `exit(1)`	The parser sets a flag, reports once, discards the line, and the loop continues
6	`echo a |` was silently accepted	The parser allowed an empty right side of `|`	Now a syntax error
7	History file written to the current directory, and read even in tests	Relative path `.history`, used in both modes	Absolute `~/.shell_history`, interactive mode only
8	`exit foo` exited anyway	Argument check warned but did not stop	Wrong argument count means the builtin does nothing
9	Zip contained three `core.*` dumps, `.o` files, and the binary	Not cleaned before packaging	`make clean`; the cores also showed crashes during development

Simplifications

- **`Command` no longer sees pipes.** It exposes `execChild(cmd)` and `runBuiltin(cmd)`; the fd wiring lives in `Pipeline.c`. About 40 lines disappeared from `Command.c`.
- **One `redirect()` helper** is used by both the in-shell builtin path and the child path, instead of two copies of the open/dup2 logic.
- **Parse tree owns its strings; `Command` copies them.** This removes the ownership agreement that used to span two modules.
- **No growable PID array.** `Pipeline` allocates one slot per command when it forks, since the count is known by then.
- **`reapJobs` loops** until no finished child remains, and `drainJobs` removes finished jobs as it goes, rather than relying on `ECHILD` to exit its loop.
- **`deq.c` rewritten around `unlinkNode`.** One function unlinks a node from either side, so `get` and `rem` are short. The file went from 149 to 118 code lines.
- **Dead code deleted:** the unused full-sequence parser (and the Makefile hack that silenced its warning) and the cached `owd` / `cwd` state.

Overall, code lines (not counting blanks or comment-only lines) went from 1061 to 927, and comment lines from 134 to 258. The module layout the handout grades on is unchanged.

Self-check questions

Try to answer each before reading the answer after the dash. If you can explain these aloud, you can defend the assignment in a demo or oral exam.

Freshman

1. What does `fork()` return in the parent and in the child? — The child's PID in the parent, and 0 in the child.
2. Why can't `cd` be run in a child process? — The working directory belongs to the process, so a child's change disappears when it exits.
3. What is a zombie? — A finished child whose exit status has not yet been collected by its parent's `wait`.

Sophomore

4. Why is `Pipeline` declared `typedef void *Pipeline`? — It hides the struct layout from other modules, so only `Pipeline.c` can depend on it.

5. Who frees the redirection file names, and why does `newCommand` copy them? — The parse tree frees its own; `Command` copies so neither module depends on the other's lifetime.
6. Why does `argv` need a NULL at the end? — `execvp` has no length argument and finds the end by looking for NULL.

Junior

7. What does `dup2(fd, 1)` do, and why do we `close(fd)` afterwards? — It makes stdout refer to the same file as `fd`; the original descriptor is then a redundant second handle.
8. A pipeline `a | cat` hangs after `a` finishes. What is the most likely bug? — Some process still holds a copy of the pipe's write end, so `cat` never sees EOF.
9. Why is the parse-then-run loop interleaved? — Later commands depend on earlier ones having run (`cd`, `exit`), so the whole line cannot be parsed and run in two separate phases.

Senior

10. Why `_exit` rather than `exit` in a forked child? — `exit` flushes inherited stdio buffers and runs `atexit` handlers that belong to the parent's state, risking duplicated output.
11. Why can't the test suite include `pwd & cd /etc ; pwd`? — One line is `/tmp` and the other `/etc`, and their order depends on scheduling.
12. What exactly happens when you press Ctrl-C during `sleep 10`, and what would you change? — The terminal sends `SIGINT` to the whole foreground process group, which includes the shell, and the shell dies. Fix: ignore the signal in the shell, give each job its own process group with `setpgid` / `tcsetpgrp`, and restore default signals in the child before `exec`.
13. How would you remove the zombie that lingers until the next Enter? — Install a `SIGCHLD` handler that sets a flag or writes to a self-pipe, and have the main loop reap when it is set, taking care to avoid the lost-wakeup race with `readline`.